

Simple Command Sequencer

Component Design Document

1 Description

The Command Sequencer component executes predefined sequences of commands. It receives high-level sequence commands and breaks them down into individual sub-commands that are sent to the command router. The sequencer handles command response tracking, timeouts, and failure modes.

2 Requirements

No requirements have been specified for this component.

3 Design

3.1 At a Glance

Below is a list of useful parameters and statistics that give a quick look into the makeup of the component.

- **Execution** - *active*
- **Number of Connectors** - 9
- **Number of Invokee Connectors** - 3
- **Number of Invoker Connectors** - 6
- **Number of Generic Connectors** - *None*
- **Number of Generic Types** - *None*
- **Number of Unconstrained Arrayed Connectors** - *None*
- **Number of Commands** - 4
- **Number of Parameters** - *None*
- **Number of Events** - 24
- **Number of Faults** - *None*
- **Number of Data Products** - 9
- **Number of Data Dependencies** - *None*
- **Number of Packets** - 1

3.2 Diagram

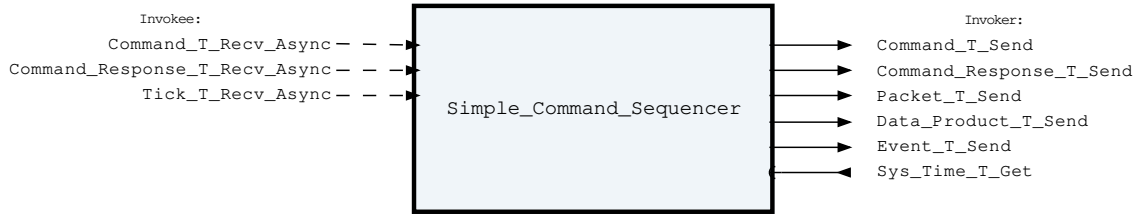


Figure 1: Simple Command Sequencer component diagram.

3.3 Connectors

Below are tables listing the component's connectors.

3.3.1 Invokee Connectors

The following is a list of the component's *invokee* connectors:

Table 1: Simple Command Sequencer Invokee Connectors

Name	Kind	Type	Return_Type	Count
Command_T_Recv_Async	recv_async	Command.T	-	1
Command_Response_T_Recv_Async	recv_async	Command_Response.T	-	1
Tick_T_Recv_Async	recv_async	Tick.T	-	1

Connector Descriptions:

- **Command_T_Recv_Async** - Sequence commands are received on this connector
- **Command_Response_T_Recv_Async** - Responses to sub-commands are received here, advancing the waiting sequence. At startup the command router sends a Register_Source message per frame via this connector with the source identifier the frame stamps into its commands. Include one arrayed connector entry per frame of either pool; a frame that never receives one can never be claimed.
- **Tick_T_Recv_Async** - Tick for managing timeouts and delays

3.3.2 Internal Queue

This component contains an internal first-in-first-out (FIFO) queue to handle asynchronous messages. This queue is sized at initialization as a configurable number of bytes. Determining the size of the component queue can be difficult. The following table lists the connectors that will put asynchronous messages onto the queue, and the maximum sizes of each of those messages on the queue. Note that each message put onto the queue also incurs an overhead on the queue of 5 additional bytes, which is included in the max message size below:

Table 2: Simple Command Sequencer Asynchronous Connectors

Name	Type	Max Size (bytes)
Command_T_Recv_Async	Command.T	265

Command_Response_T_Recv_Async	Command_Response.T	12
Tick_T_Recv_Async	Tick.T	15

If you are unsure how to size the queue of this component, it is recommended that you make the queue size a multiple of the largest size found above.

3.3.3 Invoker Connectors

The following is a list of the component's *invoker* connectors:

Table 3: Simple Command Sequencer Invoker Connectors

Name	Kind	Type	Return_Type	Count
Command_T_Send	send	Command.T	-	1
Command_Response_T_Send	send	Command_Response.T	-	1
Packet_T_Send	send	Packet.T	-	1
Data_Product_T_Send	send	Data_Product.T	-	1
Event_T_Send	send	Event.T	-	1
Sys_Time_T_Get	get	-	Sys_Time.T	1

Connector Descriptions:

- **Command_T_Send** - Sub-commands are sent out this connector
- **Command_Response_T_Send** - Sends the response to a Run_Sequence (or synthesised per-sequence) command back to the command router, either at dispatch or, for Send_After_Sequence_Completion sequences, when the sequence ends.
- **Packet_T_Send** - The packet send connector, used for sending the periodic sequencer summary packet.
- **Data_Product_T_Send** - Data products are sent out of this connector.
- **Event_T_Send** - Events are sent out of this connector
- **Sys_Time_T_Get** - The system time is retrieved via this connector.

3.4 Interrupts

This component contains no interrupts.

3.5 Initialization

Below are details on how the component should be initialized in an assembly.

3.5.1 Component Instantiation

This component contains no instantiation parameters in its discriminant.

3.5.2 Component Base Initialization

This component achieves base class initialization using the `init_Base` subprogram. This subprogram requires the following parameters:

Table 4: Simple Command Sequencer Base Initialization Parameters

Name	Type
Queue_Size	Natural

Parameter Descriptions:

- **Queue_Size** - The number of bytes that can be stored in the component's internal queue.

3.5.3 Component Set ID Bases

This component contains commands, events, packets, faults, or data products that require a base identifier to be set at initialization. The `set_Id_Bases` procedure must be called with the following parameters:

Table 5: Simple Command Sequencer Set Id Bases Parameters

Name	Type
Command_Id_Base	Command_Types.Command_Id_Base
Data_Product_Id_Base	Data_Product_Types.Data_Product_Id_Base
Event_Id_Base	Event_Types.Event_Id_Base
Packet_Id_Base	Packet_Types.Packet_Id_Base

Parameter Descriptions:

- **Command_Id_Base** - The value at which the component's command identifiers begin.
- **Data_Product_Id_Base** - The value at which the component's data product identifiers begin.
- **Event_Id_Base** - The value at which the component's event identifiers begin.
- **Packet_Id_Base** - The value at which the component's unresolved packet identifiers begin.

3.5.4 Component Map Data Dependencies

This component contains no data dependencies.

3.5.5 Component Implementation Initialization

The calling of this implementation class initialization procedure is mandatory. The component achieves implementation class initialization using the `init` subprogram. The `init` subprogram requires the following parameters:

Table 6: Simple Command Sequencer Implementation Initialization Parameters

Name	Type	Default Value
Config	Simple_Sequencer_Types. Sequencer_Config	<i>None provided</i>

Parameter Descriptions:

- **Config** - The sequencer's static configuration, exported as the `Config` constant by the generated command sequences suite package. Carries the sequence table and the sizes of the two frame pools (for sequences that wait for sub-command responses and for those that do not), which together also size the suite's generated summary packet type.

3.6 Commands

Static commands for the Simple Command Sequencer component. Per-sequence commands (one per declared sequence) are synthesised at assembly load time by `gen/models/simple_command_sequencer_commands.py`.

Table 7: Simple Command Sequencer Commands

Local ID	Command Name	Argument Type
0	Run_Sequence	Run_Sequence_Arg.T
1	Kill_All-Sequences	-
2	Kill_Frame	Packed_U16.T
3	Set_Summary_Packet_Period	Packed_U16.T

Command Descriptions:

- **Run_Sequence** - Run a command sequence by ID. The synthesised per-sequence commands are the operator-friendly form; this is the underlying backbone they all dispatch through. Response behavior is the sequence's static configuration from the sequences model.
- **Kill_All-Sequences** - Halt every running sequence and return all frames to their initial state.
Does not affect frames that were not running. Killing a sequence that is waiting for a command response does not cancel that command: its response can still arrive after the frame has been released and a new sequence has claimed it. The sequencer ignores a late response whose command id does not match the step the new sequence is waiting on, but one with a matching id would wake that step early.
- **Kill_Frame** - Halt the sequence running on a single frame and return that frame to its initial state. Fails if the frame ID is out of range; killing a frame that is not running has no effect and succeeds. Killing a frame that is waiting for a command response does not cancel that command: its response can still arrive after the frame has been released and a new sequence has claimed it. The sequencer ignores a late response whose command id does not match the step the new sequence is waiting on, but one with a matching id would wake that step early.
- **Set_Summary_Packet_Period** - Set the period of the sequencer summary packet, in ticks. A period of zero disables the packet.

3.7 Parameters

The Simple Command Sequencer component has no parameters.

3.8 Events

Events for the Simple Command Sequencer component

Table 8: Simple Command Sequencer Events

Local ID	Event Name	Parameter Type
0	Sequence_Started	Sequence_Event_Info.T
1	Sequence_Completed	Sequence_Event_Info.T
2	Sequence_Aborted	Sequence_Step_Event_Info.T
3	Sequence_Timeout	Sequence_Step_Event_Info.T

4	Sequence_Out_Of_Range_Sleep	Sequence_Sleep_Event_Info.T
5	Sequence_Out_Of_Range_Timeout	Sequence_Step_Event_Info.T
6	Command_Failure	Sequence_Step_Command_Event_Info.T
7	Invalid_Sequence_Id	Packed_U16.T
8	Invalid_Sequence_Argument_Length	Sequence_Argument_Length_Event_Info.T
9	Unexpected_Register_Source	-
10	Duplicate_Register_Source	Command_Response.T
11	No_Frame_Available	No_Frame_Available_Info.T
12	Dropped_Command	Command_Header.T
13	Dropped_Command_Response	Command_Response.T
14	Dropped_Tick	Tick.T
15	Invalid_Command_Received	Invalid_Command_Info.T
16	Unexpected_Command_Response	Command_Response.T
17	Killed_All-Sequences	-
18	Killed_Frame	Sequence_Event_Info.T
19	Invalid_Frame_Id	Packed_U16.T
20	Invalid_Dynamic_Command_Argument	Sequence_Step_Command_Event_Info.T
21	Frame_Not_Running	Packed_U16.T
22	Summary_Packet_Period_Set	Packed_U16.T
23	Invalid_Dynamic_Sleep_Argument	Sequence_Step_Event_Info.T

Event Descriptions:

- **Sequence_Started** - A sequence has begun execution on a frame
- **Sequence_Completed** - A sequence has completed all steps successfully
- **Sequence_Aborted** - A sequence was aborted due to a command failure
- **Sequence_Timeout** - A sequence timed out while waiting for a command response
- **Sequence_Out_Of_Range_Sleep** - A sleep step's wake time overflowed the system time representation, so the sequence was ended.
- **Sequence_Out_Of_Range_Timeout** - Computing a sequence's command response deadline overflowed the system time representation.
- **Command_Failure** - A command sent by a sequence received a failure response
- **Invalid_Sequence_Id** - A Run_Sequence command was received with an out of range sequence ID
- **Invalid_Sequence_Argument_Length** - A sequence command was received with an argument length that does not match the serialized length of the sequence's argument type, so the sequence was not started.
- **Unexpected_Register_Source** - A Register_Source command response was received but all sequence frames already have a source ID.
- **Duplicate_Register_Source** - A Register_Source command response carried a source ID some frame already holds, so it was refused. Responses are routed to frames by source ID alone, so frames may never share one.
- **No_Frame_Available** - A Run_Sequence command was refused because every frame of the pool its sequence draws from (waiting or non-waiting-for-response) is in use.
- **Dropped_Command** - A command was dropped due to a full queue
- **Dropped_Command_Response** - A command response was dropped due to a full queue
- **Dropped_Tick** - A tick was dropped due to a full queue

- **Invalid_Command_Received** - A command was received with invalid parameters.
- **Unexpected_Command_Response** - A command response was received with a source ID that does not belong to any active sequence frame.
- **Killed_All_Sequences** - A Kill_All_Sequences command was executed and all running sequences were halted.
- **Killed_Frame** - A Kill_Frame command was executed and the running sequence on the frame was halted.
- **Invalid_Frame_Id** - A Kill_Frame command was received with an out of range frame ID.
- **Invalid_Dynamic_Command_Argument** - A Command with a Dynamic Argument cannot be executed as the Argument is Invalid
- **Frame_Not_Running** - A Kill_Frame command targeted a frame that was not running, so there was nothing to kill.
- **Summary_Packet_Period_Set** - The summary packet period was set to a new value, in ticks.
- **Invalid_Dynamic_Sleep_Argument** - A dynamic sleep step could not resolve its duration because the sequence's argument failed validation.

3.9 Data Products

Data products for the Simple Command Sequencer component.

Table 9: Simple Command Sequencer Data Products

Local ID	Data Product Name	Type
0x0000 (0)	Frame_Running_Count	Packed_U16.T
0x0001 (1)	Frame_Running_High_Water_Mark	Packed_U16.T
0x0002 (2)	Sequences_Started_Count	Packed_U32.T
0x0003 (3)	Sequences_Finished_Count	Packed_U32.T
0x0004 (4)	Sequences_Failed_Count	Packed_U32.T
0x0005 (5)	Commands_Sent_Count	Packed_U32.T
0x0006 (6)	Last_Sequence_Started	Packed_U16.T
0x0007 (7)	Last_Sequence_Finished	Packed_U16.T
0x0008 (8)	Last_Sequence_Failed	Packed_U16.T

Data Product Descriptions:

- **Frame_Running_Count** - The number of sequence frames currently running (not idle).
- **Frame_Running_High_Water_Mark** - The maximum number of concurrently running sequence frames observed since startup.
- **Sequences_Started_Count** - The total number of sequences started since startup.
- **Sequences_Finished_Count** - The total number of sequences that ran to completion successfully since startup.
- **Sequences_Failed_Count** - The total number of sequences that ended in failure (aborted on a failed sub-command, timed out, killed, or rejected a step) since startup.
- **Commands_Sent_Count** - The total number of sub-commands the sequencer has dispatched since startup.
- **Last_Sequence_Started** - The sequence id of the most recently started sequence.
- **Last_Sequence_Finished** - The sequence id of the most recent sequence to run to completion successfully.

- **Last_Sequence_Failed** - The sequence id of the most recent sequence to end in failure.

3.10 Data Dependencies

The Simple Command Sequencer component has no data dependencies.

3.11 Packets

Packets for the Simple Command Sequencer component.

Table 10: Simple Command Sequencer Packets

Local ID	Packet Name	Type
0x0000 (0)	Summary_Packet	<i>Undefined</i>

Packet Descriptions:

- **Summary_Packet** - Periodic summary of all sequence frames, one Sequence_Frame_Summary per frame in frame order (sequence, step, state, and whether an operator awaits a deferred reply). Its type is the summary record generated by the suite the instance is initialized with. Emitted every Summary_Packet_Period ticks; zero (the default) disables it.

3.12 Faults

The Simple Command Sequencer component has no faults.

4 Unit Tests

The following section describes the unit test suites written to test the component.

4.1 *Simple_Command_Sequencer_Tests* Test Suite

This is a unit test suite for the Simple Command Sequencer component

Test Descriptions:

- **Test_Start_Sequence** - *No description provided.*
- **Test_Dynamic_Sequence** - *No description provided.*
- **Test_Invalid_Sequence_Id** - *No description provided.*
- **Test_No_Frame_Available** - *No description provided.*
- **Test_Command_Failure_Aborts_Sequence** - *No description provided.*
- **Test_Command_Failure_Continues_Sequence** - *No description provided.*
- **Test_Concurrent_Sequences** - *No description provided.*
- **Test_Spurious_Command_Response** - *No description provided.*
- **Test_Tick_Does_Not_Wake_Waiting_For_Resp** - *No description provided.*
- **Test_Dropped_Command** - *No description provided.*
- **Test_Dropped_Command_Response** - *No description provided.*
- **Test_Dropped_Tick** - *No description provided.*
- **Test_Invalid_Command** - *No description provided.*

- **Test_No_Wait_Sequence** - *No description provided.*
- **Test_Frame_Reuse_After_Completion** - *No description provided.*
- **Test_Out_Of_Range_Sleep** - *No description provided.*
- **Test_Static_Out_Of_Range_Sleep** - *No description provided.*
- **Test_Dynamic_Sleep** - *No description provided.*
- **Test_Timeout** - *No description provided.*
- **Test_Out_Of_Range_Timeout** - *No description provided.*
- **Test_Unexpected_Register_Source** - *No description provided.*
- **Test_Duplicate_Register_Source** - *No description provided.*
- **Test_Kill_All_Sequences** - *No description provided.*
- **Test_Set_Summary_Packet_Period** - *No description provided.*
- **Test_Synthesized_Sequence_Command** - *No description provided.*
- **Test_Deferred_Response_On_Completion** - *No description provided.*
- **Test_Deferred_Response_On_Abort** - *No description provided.*
- **Test_Deferred_Response_On_Timeout** - *No description provided.*
- **Test_Deferred_Response_On_Kill_All** - *No description provided.*
- **Test_Concurrent_Deferred_Responses** - *No description provided.*
- **Test_Deferred_Claim_Failure** - *No description provided.*
- **Test_Sub_Sequence_Call** - *No description provided.*
- **Test_Summary_Packet** - *No description provided.*
- **Test_Static_Deferred_Response** - *No description provided.*
- **Test_Kill_Frame** - *No description provided.*
- **Test_Data_Products** - *No description provided.*
- **Test_Response_Drives_Execution** - *No description provided.*
- **Test_Dynamic_Arg_Packing** - *No description provided.*
- **Test_Invalid_Dynamic_Command_Argument** - *No description provided.*
- **Test_Invalid_Dynamic_Sleep_Argument** - *No description provided.*
- **Test_Ghost_Command_Rejections** - *No description provided.*
- **Test_Invalid_Sequence_Argument_Length** - *No description provided.*
- **Test_Pool_Selection** - *No description provided.*
- **Test_Non_Waiting_For_Response_Pool_Exhausted** - *No description provided.*

5 Appendix

5.1 Preamble

This component contains no preamble code.

5.2 Packed Types

The following section outlines any complex data types used in the component in alphabetical order. This includes packed records and packed arrays that might be used as connector types, command arguments, event parameters, etc..

Command.T:

Generic command packet for holding arbitrary commands

Table 11: Command Packed Record : 2080 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Command_Header.T	-	40	0	39	-
Arg_Buffer	Command_Types. Command_Arg_Buffer_Type	-	2040	40	2079	Header.Arg_Buffer_Length

Field Descriptions:

- **Header** - The command header
- **Arg_Buffer** - A buffer that contains the command arguments

Command_Header.T:

Generic command header for holding arbitrary commands

Table 12: Command_Header Packed Record : 40 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_Types. Command_Source_Id	0 to 65535	16	0	15
Id	Command_Types. Command_Id	0 to 65535	16	16	31
Arg_Buffer_Length	Command_Types. Command_Arg_Buffer_Length_Type	0 to 255	8	32	39

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Id** - The command identifier
- **Arg_Buffer_Length** - The number of bytes used in the command argument buffer

Command_Response.T:

Record for holding command response data.

Table 13: Command_Response Packed Record : 56 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Source_Id	Command_Types.Command_Source_Id	0 to 65535	16	0	15

Registration_Id	Command_Types.Command_Registration_Id	0 to 65535	16	16	31
Command_Id	Command_Types.Command_Id	0 to 65535	16	32	47
Status	Command_Enums.Command_Response_Status.E	0 => Success 1 => Failure 2 => Id_Error 3 => Validation_Error 4 => Length_Error 5 => Dropped 6 => Register 7 => Register_Source	8	48	55

Field Descriptions:

- **Source_Id** - The source ID. An ID assigned to a command sending component.
- **Registration_Id** - The registration ID. An ID assigned to each registered component at initialization.
- **Command_Id** - The command ID for the command response.
- **Status** - The command execution status.

Data_Product.T:

Generic data product packet for holding arbitrary data types

Table 14: Data_Product Packed Record : 1096 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Data_Product_Header.T	-	72	0	71	-
Buffer	Data_Product_Types.Data_Product_Buffer_Type	-	1024	72	1095	Header.Buffer_Length

Field Descriptions:

- **Header** - The data product header
- **Buffer** - A buffer that contains the data product type

Data_Product_Header.T:

Generic data_product packet for holding arbitrary data_product types

Table 15: Data_Product_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Data_Product_Types.Data_Product_Id	0 to 65535	16	48	63

Buffer_Length	Data_Product_Types.Data_Product_Buffer_Length_Type	0 to 128	8	64	71
---------------	--	----------	---	----	----

Field Descriptions:

- **Time** - The timestamp for the data product item.
- **Id** - The data product identifier
- **Buffer_Length** - The number of bytes used in the data product buffer

Event.T:

Generic event packet for holding arbitrary events

Table 16: Event Packed Record : 328 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Event_Header.T	-	72	0	71	-
Param_Buffer	Event_Types.Parameter_Buffer_Type	-	256	72	327	Header.Param_Buffer_Length

Field Descriptions:

- **Header** - The event header
- **Param_Buffer** - A buffer that contains the event parameters

Event_Header.T:

Generic event packet for holding arbitrary events

Table 17: Event_Header Packed Record : 72 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Event_Types.Event_Id	0 to 65535	16	48	63
Param_Buffer_Length	Event_Types.Parameter_Buffer_Length_Type	0 to 32	8	64	71

Field Descriptions:

- **Time** - The timestamp for the event.
- **Id** - The event identifier
- **Param_Buffer_Length** - The number of bytes used in the param buffer

Invalid_Command_Info.T:

Record for holding information about an invalid command

Table 18: Invalid_Command_Info Packed Record : 112 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Id	Command_Types. Command_Id	0 to 65535	16	0	15
Errant_Field_Number	Interfaces. Unsigned_32	0 to 4294967295	32	16	47
Errant_Field	Basic_Types.Poly_ Type	-	64	48	111

Field Descriptions:

- **Id** - The command Id received.
- **Errant_Field_Number** - The field that was invalid. 1 is the first field, 0 means unknown field, 2**32 means that the length field of the command was invalid.
- **Errant_Field** - A polymorphic type containing the bad field data, or length when Errant_Field_Number is 2**32.

No_Frame_Available_Info.T:

The sequence a Run_Sequence command asked for and the frame pool that had no idle frame for it. Sequences that wait for sub-command responses and sequences that do not draw from separate pools.

Table 19: No_Frame_Available_Info Packed Record : 24 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence Id	Interfaces. Unsigned_16	0 to 65535	16	0	15
Pool	Sequence_Enums. Frame_Pool.E	0 => Waiting_For_Response_Frame 1 => Non_Waiting_For_Response_Frame	8	16	23

Field Descriptions:

- **Sequence_Id** - The ID of the sequence that could not be started
- **Pool** - The frame pool the sequence draws from, chosen by whether it waits for sub-command responses

Packed_U16.T:

Single component record for holding packed unsigned 16-bit value.

Table 20: Packed_U16 Packed Record : 16 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Interfaces. Unsigned_16	0 to 65535	16	0	15

Field Descriptions:

- **Value** - The 16-bit unsigned integer.

Packed_U32.T:

Single component record for holding packed unsigned 32-bit value.

Table 21: Packed_U32 Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Value	Interfaces. Unsigned_32	0 to 4294967295	32	0	31

Field Descriptions:

- **Value** - The 32-bit unsigned integer.

Packet.T:

Generic packet for holding arbitrary data

Table 22: Packet Packed Record : 8128 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Header	Packet_ Header.T	-	96	0	95	-
Buffer	Packet_ Types.Packet_ Buffer_Type	-	8032	96	8127	Header. Buffer_Length

Field Descriptions:

- **Header** - The packet header
- **Buffer** - A buffer that contains the packet data

Packet_Header.T:

Generic packet header for holding arbitrary data

Table 23: Packet_Header Packed Record : 96 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Id	Packet_Types. Packet_Id	0 to 65535	16	48	63
Sequence_Count	Packet_Types. Sequence_Count_Mod_ Type	0 to 16383	16	64	79
Buffer_Length	Packet_Types. Packet_Buffer_ Length_Type	0 to 1004	16	80	95

Field Descriptions:

- **Time** - The timestamp for the packet item.
- **Id** - The packet identifier
- **Sequence_Count** - Packet Sequence Count

- **Buffer_Length** - The number of bytes used in the packet buffer

Run_Sequence_Arg.T:

Argument Type for Run Sequence Command

Table 24: Run_Sequence_Arg Packed Record : 2040 bits (*maximum*)

Name	Type	Range	Size (Bits)	Start Bit	End Bit	Variable Length
Sequence_Id	Interfaces. Unsigned_16	0 to 65535	16	0	15	–
Arg_Length	Simple_ Sequencer_ Types.Run_ Sequence_ Arg_Buffer_ Length_Type	0 to 251	16	16	31	–
Buffer_Arg	Simple_ Sequencer_ Types.Run_ Sequence_ Buffer_Type	-	2008	32	2039	Arg_Length

Field Descriptions:

- **Sequence_Id** - Id of a Command Sequence Registered with the Simple Command Sequencer
- **Arg_Length** - Length of the buffer argument in bytes
- **Buffer_Arg** - Passthrough Argument for Commands in Sequence

Sequence_Argument_Length_Event_Info.T:

Sequence identification with the received and expected argument lengths for events

Table 25: Sequence_Argument_Length_Event_Info Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces. Unsigned_16	0 to 65535	16	0	15
Received_Length	Natural	0 to 2147483647	32	16	47
Expected_Length	Natural	0 to 2147483647	32	48	79

Field Descriptions:

- **Sequence_Id** - The ID of the sequence
- **Received_Length** - The argument length received with the command, in bytes
- **Expected_Length** - The serialized length of the sequence's argument type, in bytes

Sequence_Event_Info.T:

Basic sequence and frame identification for events

Table 26: Sequence_Event_Info Packed Record : 32 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces. Unsigned_16	0 to 65535	16	0	15
Frame_Id	Interfaces. Unsigned_16	0 to 65535	16	16	31

Field Descriptions:

- **Sequence_Id** - The ID of the sequence
- **Frame_Id** - The frame the sequence is executing on

Sequence_Sleep_Event_Info.T:

Sequence sleep event information

Table 27: Sequence_Sleep_Event_Info Packed Record : 64 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces. Unsigned_16	0 to 65535	16	0	15
Frame_Id	Interfaces. Unsigned_16	0 to 65535	16	16	31
Milliseconds	Natural	0 to 2147483647	32	32	63

Field Descriptions:

- **Sequence_Id** - The ID of the sequence
- **Frame_Id** - The frame the sequence is executing on
- **Milliseconds** - The number of milliseconds to sleep

Sequence_Step_Command_Event_Info.T:

Sequence, frame, step, and command identification for events

Table 28: Sequence_Step_Command_Event_Info Packed Record : 64 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces. Unsigned_16	0 to 65535	16	0	15
Frame_Id	Interfaces. Unsigned_16	0 to 65535	16	16	31
Step	Interfaces. Unsigned_16	0 to 65535	16	32	47
Command_Id	Command_Types. Command_Id	0 to 65535	16	48	63

Field Descriptions:

- **Sequence_Id** - The ID of the sequence
- **Frame_Id** - The frame the sequence is executing on
- **Step** - The step index

- **Command_Id** - The ID of the command

Sequence_Step_Event_Info.T:

Sequence, frame, and step identification for events

Table 29: Sequence_Step_Event_Info Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Sequence_Id	Interfaces. Unsigned_16	0 to 65535	16	0	15
Frame_Id	Interfaces. Unsigned_16	0 to 65535	16	16	31
Step	Interfaces. Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Sequence_Id** - The ID of the sequence
- **Frame_Id** - The frame the sequence is executing on
- **Step** - The step index

Sys_Time.T:

A record which holds a time stamp using EMA VTC format including seconds and subseconds since epoch (1-5-1980 to 1-6-1980 midnight).

Table 30: Sys_Time Packed Record : 48 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Seconds	Interfaces. Unsigned_32	0 to 4294967295	32	0	31
Subseconds	Interfaces. Unsigned_16	0 to 65535	16	32	47

Field Descriptions:

- **Seconds** - The number of seconds elapsed since epoch.
- **Subseconds** - The number of $1/(2^{16})$ sub-seconds.

Tick.T:

The tick datatype used for periodic scheduling. Included in this type is the Time associated with a tick and a count.

Table 31: Tick Packed Record : 80 bits

Name	Type	Range	Size (Bits)	Start Bit	End Bit
Time	Sys_Time.T	-	48	0	47
Count	Interfaces. Unsigned_32	0 to 4294967295	32	48	79

Field Descriptions:

- **Time** - The timestamp associated with the tick.
- **Count** - The cycle number of the tick.

5.3 Enumerations

The following section outlines any enumerations used in the component.

Command_Enums.Command_Response_Status.E:

This status enumeration provides information on the success/failure of a command through the command response connector.

Table 32: Command_Response_Status Literals:

Name	Value	Description
Success	0	Command was passed to the handler and successfully executed.
Failure	1	Command was passed to the handler not successfully executed.
Id_Error	2	Command id was not valid.
Validation_Error	3	Command parameters were not successfully validated.
Length_Error	4	Command length was not correct.
Dropped	5	Command overflowed a component queue and was dropped.
Register	6	This status is used to register a command with the command routing system.
Register_Source	7	This status is used to register command sender's source id with the command router for command response forwarding.

Sequence_Enums.Frame_Pool.E:

The two disjoint sets of sequence frames, split by whether the sequences they run wait for their sub-commands' responses. Keeping the pools separate means a response still in flight to a finished non-waiting-for-response run can never wake a frame parked waiting for a response. Frame ids are global across both pools, with the waiting-for-response frames first.

Table 33: Frame_Pool Literals:

Name	Value	Description
Waiting_For_Response_Frame	0	A frame for sequences declared with <code>wait_for_command_completion</code> true. The frame parks after each command step until that sub-command's response arrives or its timeout expires. The pool holds <code>num_waiting_for_response_frames</code> frames.

Non_Waiting_For_Response_Frame	1	A frame for sequences declared with <code>wait_for_command_completion</code> false. Command steps are dispatched without awaiting their responses, though the frame still parks for sleep steps. The pool holds <code>num_non_waiting_for_response_frames</code> frames.
--------------------------------	---	--